

Predictive KV Cache Amortization for Edge Language Model Inference on Mobile Devices

Arjit Tripathi

arjit.tripathi2023@vitstudent.ac.in

Medha Sriram

medha.sriram2023@vitstudent.ac.in

Heer Shah

shah.heer2023@vitstudent.ac.in

Dr. Shalini L

l.shalini@vit.ac.in

Abstract—A language model running on a phone often needs a description of the user’s situation (where they are, who is with them, what time of day it is) before it can answer usefully. Feeding that description in with every request is expensive: on our test phone, the prefill step for a 439-token prompt took 14.2 s. Server systems avoid repeated prefill by reusing cached key-value (KV) state across requests, but that depends on many users sending overlapping prompts. A single phone has no such request stream. It does, however, have sensors. We present CAP-KVC, which uses GPS, Bluetooth Low Energy (BLE) proximity and time of day to rank a small set of pre-written contexts, prefills the likely ones while the device is idle, and keeps at most three of them resident as llama.cpp KV state. At interaction time only the user’s intent tokens are decoded. On a OnePlus 11R running Qwen2.5-0.5B-Instruct (Q4_K_M), resuming a restored 44-token context took 351 ms of prefill versus 2507 ms for decoding the same context inline ($7.1\times$, $n = 30$ each); restoring the state took 2.4 ms. Over 100 synthetic days of sensor traces with five contexts, the deployed routing and residency rules used the correct context for 98.5% of 6,400 interactions, against 92.2% for a request-driven cache without sensing, which misses on every first visit. We also report what this costs: 1.5% of interactions used the wrong context, and an attacker who can forge GPS without detection raised that to 38.9%. The evaluation covers one device, one model, five contexts and synthetic traces.

Index Terms—Key-value cache, edge inference, predictive caching, language models, mobile systems, sensor-driven context.

I. INTRODUCTION

Small instruction-tuned models now run entirely on phones once quantized to 4 bits [4], [5], [2]. For many assistive uses the model is not useful on the user’s words alone. An augmentative and alternative communication (AAC) app [1], for instance, receives a short intent such as “I need water” and is expected to expand it into something that fits the setting: a hospital ward at 11 a.m. is different from home in the evening with a caregiver present. The usual way to provide this is to put a context description in front of the intent and let the model read both.

Reading the context is the prefill step: one forward pass over all prompt tokens before the first output token appears. Its cost grows with prompt length. On the phone used in this paper, prefill took 2.5 s for a 52-token prompt and 14.2 s for 439 tokens (Section V-B), roughly 30 ms per token. For a user who communicates through the device, that delay sits in the middle of every turn.

Datacenter serving systems handle the same problem by not recomputing prefixes they have already seen. SGLang keeps prompt prefixes in a radix tree [6], vLLM shares KV blocks between requests [7], Prompt Cache stores attention states for reusable prompt modules [8], and RAGCache caches the KV state of retrieved documents across requests [9]. All of these are request-driven: the cache is filled when a request containing the prefix arrives, and it pays off when later requests share that prefix. That works on a server handling many users. On a phone used by one person, the request that would populate the cache is the user’s own first turn in a new setting, and that turn pays the full prefill cost.

A phone does have information a server lacks: its sensors usually reveal the setting before the user says anything. CAP-KVC (Context-Aware Predictive KV-Cache priming) uses this. A small set of contexts is written in advance. A background sweep scores them against GPS position, nearby BLE devices and time of day, prefills the highest-ranked ones while the device is idle, and keeps their KV state resident. When the user acts, the app resumes from the best resident context and decodes only the intent. The hypothesis we test is that sensor readings predict the needed context often enough for this to beat request-driven reuse, at an acceptable rate of wrong-context answers.

This paper makes three contributions:

- 1) A sensor-driven predictive KV caching design for a single user: deterministic context scoring, a bounded resident set with hysteresis-gated replacement, and a two-tier fallback when no relevant state is resident.
- 2) An Android implementation on llama.cpp with per-slot KV residency, rollback of each interaction so a resident state is reused unchanged, crash-safe cache persistence, and location input validation.
- 3) An evaluation that separates the restore benefit (matched-context latency) from prediction quality (trace-driven comparison against oracle, request-driven, random and single-sensor baselines) and from adversarial sensor input.

II. RELATED WORK

Reusing KV state across requests. RadixAttention in SGLang [6] matches incoming prompts against a radix tree of cached prefixes and evicts by LRU. PagedAttention [7]

TABLE I
WHERE CAP-KVC DIFFERS FROM REQUEST-DRIVEN KV REUSE.

	Request-driven reuse	CAP-KVC
Setting	Multi-user server	One user, one phone
Cache filled	When a request arrives	Before it, while idle
Selection signal	Prompt prefix / document	GPS, BLE, time
First request in a context	Full prefill	Restore if predicted
Failure mode	Cold miss	Wrong context or cold miss

lets requests share KV blocks. Prompt Cache [8] precomputes attention states for prompt modules that applications reuse, and CacheBlend [10] combines cached chunks that are not prefixes. RAGCache [9] caches the KV state of retrieved documents in a GPU/host hierarchy and replaces entries with a policy that accounts for retrieval patterns and recomputation cost, outperforming LRU. CacheGen [11] compresses KV state for network transfer. These systems decide what to keep after seeing requests. CAP-KVC decides what to compute before the request exists, from sensor evidence, which is the only signal available ahead of the first request on a single-user device (Table I). We do not run SGLang or RAGCache themselves; they target GPU servers. Section V-C instead simulates their request-driven policy (LRU over contexts identified at request time) on the same traces.

Shrinking the KV cache. H₂O [12], Scissorhands [13], SnapKV [14], StreamingLLM [15] and KVQuant [16] reduce KV memory within a sequence. They are complementary: a compressed state can still be primed and restored.

On-device inference. Quantization [4], [5], small-model design [17], mobile runtimes [2], [18], flash offloading [19] and heterogeneous NPU/CPU execution [20] make phone inference feasible. They lower the cost of each token but do not remove the prefill of a long context from the interaction path.

III. SYSTEM DESIGN

Fig. 1 shows the components. A background sweep reads sensors, ranks contexts, and updates the resident set. The interaction path picks a resident state and decodes the intent.

A. Problem

Let $C = \{c_1, \dots, c_M\}$ be pre-written contexts with prompts p_i of N_i tokens. Inline (retrieval-augmented style) inference decodes p_i followed by the intent q on every request, so prefill costs grow with $N_i + |q|$. CAP-KVC moves the prefill of p_i to idle time and stores its KV state; at interaction time it restores the state and decodes only q . The open question is which p_i to prepare, since memory allows only $K < M$ resident states. The prototype has $M = 5$ contexts (home morning, home evening, hospital ward, therapy room, caregiver visit) and $K = 3$.

B. Context Scoring

Each context c_i has an expected hour t_i , an anchor location ℓ_i and a set of BLE anchors A_i with weights w_{ij} . For a sensor snapshot the score is

$$S(c_i) = \alpha S_{\text{time}} + \beta S_{\text{gps}} + \gamma S_{\text{ble}}, \quad \alpha + \beta + \gamma = 1, \quad (1)$$

with

$$S_{\text{time}} = \exp(-d_{\text{circ}}(t, t_i)^2 / 2\sigma^2), \quad \sigma = 2 \text{ h}, \quad (2)$$

$$S_{\text{gps}} = \exp(-d_{\text{hav}}(\ell, \ell_i) / \lambda), \quad \lambda = 0.1 \text{ km}, \quad (3)$$

$$S_{\text{ble}} = \sum_{j \in A_i} w_{ij} \mathbb{I}[j \text{ seen}] / \sum_{j \in A_i} w_{ij}. \quad (4)$$

d_{circ} wraps around midnight, so S_{gps} falls to 0.37 at 100 m. The weights come from per-sensor reliability, normalized to sum to one: time has a fixed weight $w_t = 0.4$; GPS has $w_g = \exp(-a/50 \text{ m})$ for reported accuracy a , or 0 without a fix; BLE has w_b equal to the mean of $1/(1 + e^{-0.1(r+70)})$ over observed RSSI values r , or 0 if nothing is seen. Contexts scoring below 0.01 are dropped. The scorer has no learned parameters; “predictive” refers to preparing a context before the request, not to a trained model.

C. Residency and Hysteresis

The resident set holds at most $K = 3$ KV states. After each sweep, candidates are taken in rank order. A candidate already resident is kept; otherwise it takes a free slot, or replaces the weakest resident state if its score exceeds that state’s score by more than Δ . Δ is a hysteresis margin on scores. It is unrelated to K ; the submitted version of this paper conflated the two and justified Δ with an unrelated text-similarity experiment, which we drop. The deployed value is $\Delta = 0.10$; Section V-D measures its effect. The same rule runs in the 60-second sweep and in a 15-minute background check that sees no BLE data and may only add the top-ranked context.

D. Interaction and Fallback

At interaction time the app uses the highest-ranked context that is resident. If none is, the interaction is a cold miss: a canned response is shown immediately (Tier 1) and priming of the current top contexts starts in the background (Tier 2). A wrong-context answer happens when a resident context is used but is not the true one.

IV. IMPLEMENTATION

AACBridge is an Android app (Kotlin, API 26+, arm64-v8a) using llama.cpp [2] through JNI, with Qwen2.5-0.5B-Instruct [3] quantized to Q4_K_M.

Native boundary. The JNI layer exposes 14 functions, all with scalar arguments and results: lifecycle (initialize backend, load model, release), KV operations (save, load, clear scratch, reset slot), inference (inline run, prefill only, resume) and four timing getters. A single `ReentrantLock` serializes every native call, because llama.cpp context state is shared.

Sequence slots. The llama.cpp context is created with four sequences of 1024 tokens: three for resident states and one scratch sequence for inline inference and benchmark restores,

Background: sweep every 60s; BLE-free check every 15 min

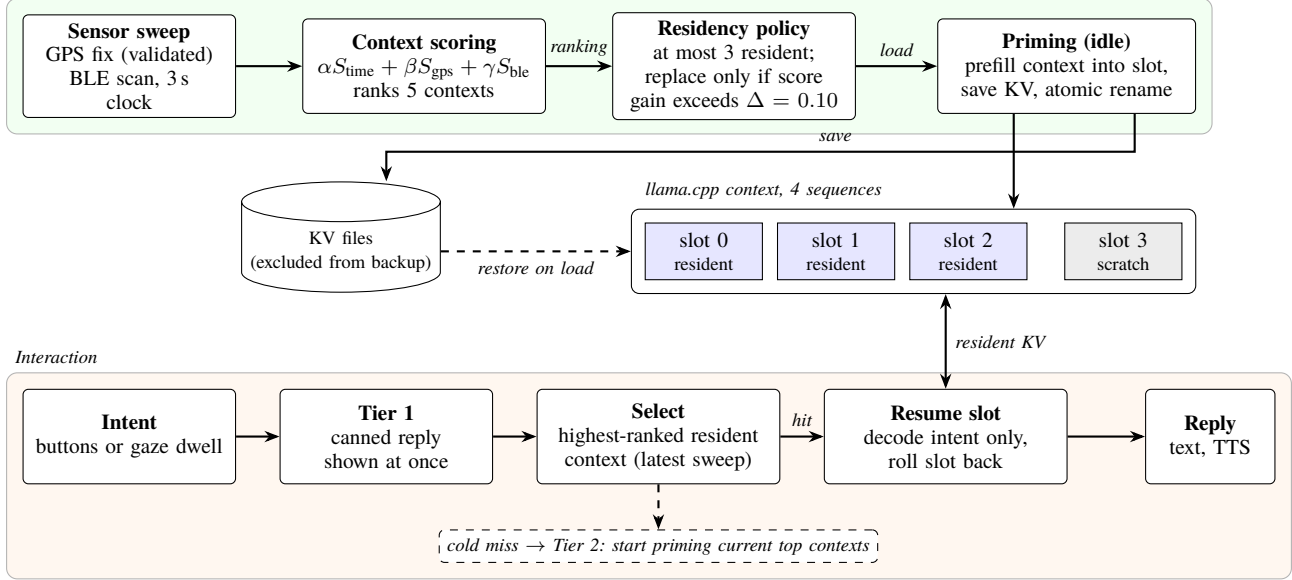


Fig. 1. CAP-KVC architecture. A background sweep scores five pre-written contexts from GPS, BLE and time, keeps up to three of them as primed KV state in llama.cpp sequence slots, and saves each state to disk. An interaction shows a canned reply, then resumes the highest-ranked resident context and decodes only the intent; if none is resident, priming starts in the background. Slot 3 serves inline inference without disturbing resident states.

so neither disturbs a resident state. Priming decodes a context directly into the slot assigned to it and serializes that slot to a temporary file, which is then renamed into place, so a crash cannot leave a partial cache file. Resuming appends the intent at the slot’s current position and, after generation, removes everything after the cached context, so the resident state is reused unchanged by the next interaction. With 24 layers, 2 KV heads of dimension 64 and fp16 storage, Qwen2.5-0.5B needs 12 KiB of KV per token; the four sequences reserve 48 MiB at context creation. We did not re-measure process memory after this layout change.

Sensing. A foreground service runs the sweep every 60 s: a 3-second BLE scan and a location fix. Each provider’s last fix is validated (not flagged as mock by the OS, at most 120 s old, reported accuracy between 2 and 500 m); if none passes, one fresh fix is requested, preferring the network provider over GPS. A WorkManager job runs every 15 minutes without BLE scanning.

Intent input. Intents come from on-screen buttons or a gaze-dwell interface; an sEMG+gaze fusion classifier from earlier work is included as a prototype input but is not evaluated here.

Backup. The manifest disables app backup and excludes the KV cache directory from cloud backup and device-to-device transfer.

V. EVALUATION

A. Setup

Latency was measured on a OnePlus 11R (Snapdragon 8+ Gen 1, 8 GB RAM, Android 14) with 4 threads, batch 512, greedy decoding and at most 64 output tokens.

TABLE II
NATIVELY TIMED LATENCY COMPONENTS (MS), MEDIAN [IQR], $n = 30$ EACH.
CAP-KVC RESTORES A 44-TOKEN CONTEXT AND DECODES THE 9 INTENT TOKENS.

Condition (tokens)	Prefill	Generation	Restore
No context (9)	377 [288, 503]	5775 [2823, 6213]	–
Inline, matched (52)	2449 [2231, 2777]	7964 [7350, 9374]	–
Inline (89)	4116 [3677, 4487]	8589 [7822, 11294]	–
Inline (206)	7075 [5148, 8700]	6991 [4107, 10275]	–
Inline (439)	14024 [13219, 14941]	5912 [5467, 7003]	–
CAP-KVC (9)	331 [276, 355]	4395 [3985, 5110]	1.2 [1.0, 3.3]

Each condition ran 2 warm-up and 30 measured trials. Prefill and generation times come from `std::chrono` timestamps around `llama_decode` in native code; restore time is measured around the load call. All 180 trials and the analysis script are in the repository.

During this run the phone’s gaze interface remained active and issued 45 interaction inferences between trials. These held the engine lock, and the trial timer started before the lock was taken, so the end-to-end totals of some trials include waiting time. We therefore report only the natively timed components, which the lock wait does not affect, and not end-to-end totals. The app now blocks interaction inference during benchmarks and starts trial timers after taking the lock.

B. Matched-Context Latency

The CAP-KVC cache holds a 44-token context. The matched inline condition decodes the same context plus the intent (52 tokens). Larger inline conditions use longer contexts and show how inline prefill scales; their ratios to CAP-KVC grow by construction and are not matched comparisons.

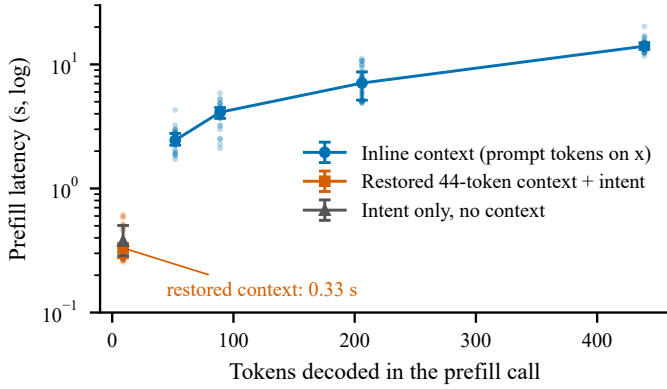


Fig. 2. Prefill latency against tokens decoded (log scale). Points are trials; markers show median and IQR ($n = 30$ per condition).

With the context matched, mean prefill fell from 2507 ms (SD 522) to 351 ms (SD 97), a ratio of 7.1 (Welch $t(31.0) = 22.2$, $p < 10^{-19}$; Mann–Whitney $p < 10^{-10}$; Table II). Restoring the state took 2.4 ms on average (median 1.2 ms). A rough time to first token, taken as restore plus prefill plus one mean generation step, is about 0.43 s for CAP-KVC and 2.6 s inline. Inline prefill grew by about 30 ms per prompt token (OLS, $R^2 = 0.91$; Fig. 2), reaching 14.2 s at 439 tokens, where the CAP-KVC prefill was $40.5\times$ shorter. That larger ratio compares different context lengths and reflects the cost CAP-KVC avoids when contexts are long, not a matched speedup. We measured restore only for the 44-token cache, so we cannot say how restore time grows with context length.

Generation dominates the full response. With 64 output tokens, generation took 4.4–8.6 s (medians), so the 2.1 s saved in prefill at matched length shortens the complete response far less than the prefill ratio suggests. The benefit is concentrated in time to first token. Generation times varied between conditions and trials (IQRs above 1 s); the run recorded no thermal or CPU-frequency data, so we cannot attribute this variation.

C. Prediction and Cache Policy

Traces. We have no recorded sensor traces from users, so we use synthetic ones (seed 42). Each of 100 days follows the five-context routine 06:00–22:00, with every context boundary shifted per day by $U(\pm 0.5 \text{ h})$. A sweep runs every minute and an interaction every 15 minutes (6,400 interactions). Each sweep draws GPS displacement $\mathcal{N}(0, 50 \text{ m})$, GPS dropout 10%, reported accuracy $\mathcal{N}(15, 10) \text{ m}$, per-beacon detection 80% with RSSI $\mathcal{N}(-65, 10) \text{ dBm}$, BLE outage 15%, and a spurious beacon 10% of the time. The high-noise setting uses $\pm 1.5 \text{ h}$ shifts, 200 m GPS noise, 30% GPS dropout, 50 m accuracy, 50% detection and 40% BLE outage. The context time anchors were written from the same routine, so time-of-day evidence is favourable by construction.

Simulation. The evaluator calls the production scoring, residency and selection code. It does not model priming delay: a state chosen at one sweep is assumed usable by the next minute.

TABLE III
INTERACTION OUTCOMES OVER 6,400 SYNTHETIC INTERACTIONS (%) AND STATES LOADED PER DAY. $\Delta = 0.10$ UNLESS NOTED.

Policy / scenario	Correct	Wrong	Cold	Loads/d
Oracle (always resident)	100.0	0.0	0.0	5.0
Request-driven LRU, no sensing	92.2	0.0	7.8	5.0
Random ranking	20.7	79.3	0.0	1011
Time only	92.2	7.8	0.0	5.0
GPS only	87.9	12.1	0.0	45.7
BLE only	92.6	6.9	0.5	115.8
All sensors (deployed)	98.5	1.5	0.0	31.6
All sensors, $\Delta = 0$	98.5	1.5	0.0	65.9
GPS 70% unavailable	98.2	1.8	0.0	116.7
High noise	90.6	9.4	0.0	106.2
GPS forged, not detected	61.1	38.9	0.0	18.2
GPS forged, mock flag rejected	98.3	1.7	0.0	75.8
Cloned BLE beacons	90.1	9.9	0.0	103.2

Metrics. For each interaction with true context c^* and used context $\hat{c}$: correct if $\hat{c} = c^*$; wrong context if $\hat{c} \neq c^*$; cold miss if no ranked context is resident. These three partition the interactions. We also report loads per day (states primed or restored into the resident set), which measures churn.

Baselines. *Oracle*: c^* is always resident, giving the restore benefit with perfect prediction. *Request-driven LRU*: no sensing; the context becomes known when the request arrives (the best case for request-driven reuse), is used if cached, and is otherwise a cold miss and inserted into a three-entry LRU cache that persists across days. *Random*: uniform random scores each sweep, same residency rule. *Single sensor*: time only (no GPS or BLE), and GPS only or BLE only with w_t reduced to 0.001 so that time does not leak in through normalization.

Table III shows the results. With all sensors the deployed rules used the correct context for 98.5% of interactions. The request-driven LRU baseline never used a wrong context but missed 7.8% of interactions, one at each of the five daily context changes, because five contexts visited in order never fit in three entries. Sensing removed those misses at the price of 1.5% wrong-context interactions. Each sensor alone did worse than the combination: GPS cannot separate the three contexts that share the home coordinates, and with BLE alone the router produced no candidate in 0.5% of interactions, when no registered beacon was seen. Time alone reached 92.2%, which reflects how the traces were built rather than a property of real schedules.

Cold misses were near zero for every sensing policy, and in every sensing configuration the router’s top-1 accuracy equalled the share of correct interactions. With three of five contexts resident, the dominant failure is a wrong context rather than a miss, so Hit@3-style residency metrics say little in this configuration. Random ranking also had no cold misses but was correct only 20.7% of the time.

D. Hysteresis

Fig. 3 varies Δ with everything else fixed. Raising Δ from 0 to the deployed 0.10 halved loads per day under normal

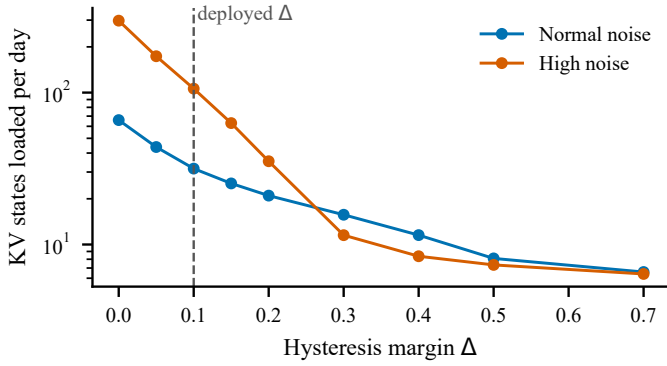


Fig. 3. States loaded per day against hysteresis margin Δ (100 synthetic days each). The share of correct interactions stayed at 98.5% (normal) and 90.6% (high noise) across all margins up to 0.5.

noise (65.9 to 31.6) and cut them from 297.5 to 106.2 under high noise. The share of correct interactions did not change for any margin up to 0.5, presumably because a change of context moves scores by more than these margins. Each load is a restore or, if the cache file is missing, a full prefill, so the margin mainly saves background work. On these traces a margin of 0.3 would cut loads further at no cost in accuracy; we keep 0.10 until real traces are available, since a larger margin also delays switching after a genuine but small change in evidence.

E. Not Measured

We did not measure battery drain, power, temperature or CPU frequency for the 60-second sweep, the 15-minute job or background priming. Each load in Table III is a unit of background work whose energy cost remains to be quantified on the device.

VI. THREAT MODEL

Assets. The choice of context that shapes what the device says for its user, and the cached KV files, which encode the context prompts.

Adversaries. (A) An app on the same phone supplying mock locations. (B) Someone nearby advertising BLE beacons that copy registered anchors. (C) Someone with access to the phone’s files or backups. Ordinary sensor noise and stale fixes are covered by the evaluation above.

What an attacker can do. The router only ranks the five pre-written contexts. Sensor manipulation can therefore make the app use the wrong one of them, but it cannot add a context, change a prompt, or alter the user’s intent. This bounds the damage without removing it: a wrong context still changes what the model writes for the user.

Mitigations and residual risk. Location fixes flagged as mock by Android are rejected, as are stale fixes and implausible accuracy values. When the mock flag is set, forged GPS had little effect (1.7% wrong context, Table III). On a rooted phone a forged fix need not carry the flag; if it also reports a plausible accuracy such as 5 m, the checks accept it, and the wrong-context rate rose to 38.9%. BLE advertisements

are unauthenticated and we have no defence against cloned beacons (9.9% wrong context); the router has no check that GPS and BLE agree. Cache files are stored unencrypted in app-private storage and are excluded from backup and device transfer, which protects them only as far as the Android sandbox does.

Out of scope. Rooted or compromised operating systems, kernel or firmware attackers, and physical extraction of storage.

VII. LIMITATIONS AND CONCLUSION

The latency results come from one phone, one 0.5B model and 30 trials per condition, with a single cached context size; restore time for longer contexts, end-to-end latency without interference, memory use under the current slot layout, and an independent llama.cpp throughput benchmark on the same phone remain to be measured. The prediction results come from synthetic traces whose schedule matches the contexts’ time anchors and do not model priming delay; real multi-day sensor recordings are needed to confirm them. The prototype has five hand-written contexts. Energy cost is unmeasured. Nothing here has been evaluated with AAC users, and no clinical claim is made.

Within those limits, the measurements separate the parts of the question. Restoring a primed context removed most of the matched prefill cost (351 versus 2507 ms, 2.4 ms restore). Sensor-driven residency used the correct context for 98.5% of synthetic interactions, compared with 92.2% for request-driven LRU, by trading first-visit misses for 1.5% wrong-context answers. Undetected GPS forgery is the main weakness: it raised wrong-context answers to 38.9%.

The code, benchmark logs, evaluation harness and figure scripts are available at <https://github.com/omni-ar/AACBridge>.

ACKNOWLEDGMENT

The authors used Claude (Anthropic) to draft the text of this revised manuscript (all sections), which the authors then edited; to help write and revise code in the Android application, the JNI layer, the trace evaluator and the analysis and plotting scripts used for Tables II–III and Figs. 2–3; and for language editing. Google Antigravity was used as an AI-assisted development tool for parts of the Android/JNI implementation (Section IV). Fig. 1 is drawn with TikZ code, written with Claude’s assistance from a component layout specified by the authors; Figs. 2–3 are rendered from measured or simulated data by scripts in the repository. No figure was produced by an image-generation model. Research questions, system design decisions, experiment choices and the interpretation of results are the authors’.

REFERENCES

- [1] D. R. Beukelman and J. C. Light, *Augmentative and Alternative Communication: Supporting Children and Adults with Complex Communication Needs*, 5th ed. Baltimore, MD, USA: Paul H. Brookes, 2020.
- [2] G. Gerganov *et al.*, “llama.cpp: LLM inference in C/C++,” 2023. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [3] A. Yang *et al.*, “Qwen2.5 technical report,” *arXiv:2412.15115*, 2024.

- [4] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Proc. NeurIPS*, 2022.
- [5] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *Proc. ICLR*, 2023.
- [6] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” in *Proc. NeurIPS*, 2024.
- [7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. SOSP*, 2023.
- [8] I. Gim, G. Chen, S.-S. Lee, N. Sarda, A. Khandelwal, and L. Zhong, “Prompt Cache: Modular attention reuse for low-latency inference,” in *Proc. MLSys*, 2024.
- [9] C. Jin, Z. Zhang, X. Jiang, F. Liu, S. Liu, X. Liu, and X. Jin, “RAG-Cache: Efficient knowledge caching for retrieval-augmented generation,” *ACM Trans. Comput. Syst.*, vol. 44, no. 1, Art. 2, Feb. 2026, doi: 10.1145/3768628.
- [10] J. Yao *et al.*, “CacheBlend: Fast large language model serving for RAG with cached knowledge fusion,” in *Proc. EuroSys*, 2025.
- [11] Y. Liu *et al.*, “CacheGen: KV cache compression and streaming for fast large language model serving,” in *Proc. ACM SIGCOMM*, 2024.
- [12] Z. Zhang *et al.*, “H₂O: Heavy-hitter oracle for efficient generative inference of large language models,” in *Proc. NeurIPS*, 2023.
- [13] Z. Liu *et al.*, “Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time,” in *Proc. NeurIPS*, 2023.
- [14] Y. Li *et al.*, “SnapKV: LLM knows what you are looking for before generation,” in *Proc. NeurIPS*, 2024.
- [15] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *Proc. ICLR*, 2024.
- [16] C. Hooper *et al.*, “KVQuant: Towards 10 million context length LLM inference with KV cache quantization,” in *Proc. NeurIPS*, 2024.
- [17] Z. Liu *et al.*, “MobileLLM: Optimizing sub-billion parameter language models for on-device use cases,” in *Proc. ICML*, 2024.
- [18] MLC Team, “MLC LLM,” 2023. [Online]. Available: <https://github.com/mlc-ai/mlc-llm>
- [19] K. Alizadeh *et al.*, “LLM in a flash: Efficient large language model inference with limited memory,” in *Proc. ACL*, 2024.
- [20] Z. Xue *et al.*, “PowerInfer-2: Fast large language model inference on a smartphone,” *arXiv:2406.06282*, 2024.